NATIONAL BOARD OF EXAMINATIONS — OFFICIAL ANSWER BOOKLET

CANDIDATE ROLL NO: **21CS114** COURSE CODE: **CS301**

DEGREE: B.E. (Computer Science & Engg) DATE OF EXAM: 12-NOV-2025

TOTAL PAGES WRITTEN: 03 EXAM CENTER: #402 HALL-A

Ans 1: Merge Sort Recurrence Relation & Complexity Analysis

Merge sort works on the **Divide and Conquer** design paradigm:

1. **Divide:** Split array A[low..high] at mid = (low + high) / 2 into two equal halves.
2. **Conquer:** Recursively sort subarray A[low..mid] and A[mid+1..high].
3. **Combine:** Merge the two sorted subarrays into a single sorted output.

Recurrence Relation:

> **T(n) = 2 T(n / 2) + Θ(n) [for n > 1]**
> **T(1) = Θ(1) [base case]**

Applying Master's Theorem for divide-and-conquer: T(n) = a T(n/b) + f(n)
Here a = 2, b = 2, f(n) = n^1
log_b(a) = log_2(2) = 1. Since f(n) = Θ(n^(log_b a)) = Θ(n^1),
Case 2 of Master Theorem applies:

> **⇒ T(n) = Θ(n^(log_b a) * log n) = Θ(n log n)**

Step-by-Step Split and Merge Trace on [38, 27, 43, 3, 9, 82, 10, 19]:

```
Level 0:                [38, 27, 43, 3, 9, 82, 10, 19]
```

```
                                /                   \
Level 1:            [38, 27, 43, 3]             [9, 82, 10, 19]
                       /        \                  /          \
Level 2:           [38, 27]   [43, 3]          [9, 82]     [10, 19]
                    /    \     /   \            /   \        /    \
Level 3:          [38]  [27]  [43] [3]        [9]  [82]    [10]  [19]
                  ------------------------------------------------
Merge Level 2:     [27, 38]   [3, 43]          [9, 82]     [10, 19]
Merge Level 1:        [3, 27, 38, 43]             [9, 10, 19, 82]
Final Sorted:            [3, 9, 10, 19, 27, 38, 43, 82] (Sorted)
```

Auxiliary Space Complexity: O(n) required for temporary merge buffers.

AUTHENTIC SCRIPT SEAL #EXP-2025-CS301-P1 • CANDIDATE SIGN: [VERIFIED]

Ans 2: AVL Tree Construction & Balance Factor (BF) Analysis

An AVL Tree is a Self-Balancing Binary Search Tree where balance factor condition holds for all nodes:

> **BF(Node) = Height(Left_Subtree) - Height(Right_Subtree) ∈ {-1, 0, +1}**

Sequential Insertion of keys: [15, 20, 24, 10, 13, 7, 30, 36, 25]:

- **Insert 15, 20:** Root=15 (BF = -1), Right=20 (BF = 0). Tree is balanced.
- **Insert 24:** Node 15 has BF = -2 (Right-heavy). **RR Imbalance at Node 15.**
  ⇒ **Apply Left Rotation (LL) on Node 15:**
  New Subtree Root = 20, Left child = 15 (BF = 0), Right child = 24 (BF = 0).
- **Insert 10, 13:** Node 15 Left child = 10, 10 Right child = 13. Node 15 BF = +2, Node 10 BF = -1.
  LR Imbalance at Node 15:
  ⇒ **Apply Double Rotation:** Left rotate on 10 ⇒ Right rotate on 15.
  Subtree root becomes 13, left = 10, right = 15.

```
Final Balanced AVL Tree Representation:
                [ 20 ]                (BF = 0)
              /       \
           [ 13 ]      [ 30 ]         (BF = 0)
          /      \    /      \
        [ 10 ]  [ 15][ 24 ]  [ 36 ]   (BF = 0)
        /               \
      [ 7 ]             [ 25 ]
```

Every node satisfies |BF| ≤ 1. Search, insertion, and deletion worst-case time = O(log n).

AUTHENTIC SCRIPT SEAL #EXP-2025-CS301-P2 • CANDIDATE SIGN: [VERIFIED]

Ans 3: Dijkstra's Single-Source Shortest Path Algorithm

Constraint: Edge weights w(u, v) ≥ 0 (strictly non-negative).

Algorithm using Min-Heap Priority Queue Q:

```
1. Initialize dist[source] = 0, and dist[v] = ∞ for all v ∈ V \ {source}.
2. Insert all vertices into Min-Priority Queue Q keyed by dist[v].
3. while Q is not empty:
    u = Extract_Min(Q)
    for each adjacent neighbor v of u:
        if dist[u] + w(u, v) < dist[v]:
            dist[v] = dist[u] + w(u, v) // Edge Relaxation
            Decrease_Key(Q, v, dist[v])
```

Time Complexity Analysis:

- Extract_Min called |V| times: O(V log V)
- Decrease_Key called |E| times: O(E log V)
- Total Worst-Case Time: **O((V + E) log V)** with Binary Min-Heap.
- Space Complexity: O(V) for distance table and heap pointers.

AUTHENTIC SCRIPT SEAL #EXP-2025-CS301-P3 • CONTROLLER SIGNED [END OF BOOKLET]